Lotse-RL: Executable Eligibility Rules as Verifiable Ground Truth for Benefits-Navigation Agents

Julian Schmidt
Attention Seekers (KI Challenge)
https://github.com/Julian-AT/sv-lotse

Preprint, November 6, 2026

Abstract

Assistants that guide people through public benefits must name the right benefits, office and documents, ask few questions and never give medical advice. Free-form generation guarantees none of this, and expert labels are scarce. We present SV KI-Lotse, a voice-first assistant for Austrian social insurance in which eligibility criteria, office rules and document requirements are cited, executable data evaluated by a deterministic process engine; the language model only maps the user's words to slot values and phrases the engine's next question. Because the engine computes the correct outcome of any counselling dialogue, it doubles as a verifier. We use it to build AT-Benefits, an interactive gym on the product's real tool contract whose gold plans and rewards are computed by code, and to release engine-labelled data: 1,000,000 situations, 10,352 tool-exact dialogues, 449 documents, 15,120 filled forms and a corpus of 2,863 sections of Austrian social-insurance law. On the product, the guard stack passes all 500 red-team cases with 0 medical-boundary violations and retrieval reaches recall@10 of 96.00%, yet end-to-end scenarios succeed in only 1 of 29 judged runs: the rules and components hold, the assembled agent does not yet. We fine-tune Qwen3-8B with LoRA on 883 engine-verified examples (validation loss 0.0359); its task performance has not been evaluated. Frontier-model baselines in the gym (T1) were still running when this version was built and are reported as pending. A QAOA study on IBM hardware finds no quantum advantage. Every number in this paper is generated from result files.

1 Introduction

Social insurance in Austria is spread across health, pension and accident insurance carriers, federal agencies, nine federal states and municipalities. A person whose parent can no longer live alone may be entitled to a care allowance from the pension carrier, counselling and mobile services from a state agency, care leave compensated by a federal agency and a subsidy for 24-hour care. Each benefit has its own criteria, office, documents and form, and the office often depends on insurance status and residence. The people who most need help with this, older adults, relatives acting as helpers and people with limited German, are the least able to find it.

Language models explain such procedures fluently, but a free-form answer can invent an eligibility rule, ask questions whose answers do not matter, or drift into medical advice. A public-sector assistant must avoid all three: the challenge brief behind this work forbids diagnoses, interpretation of findings, therapy decisions and medication advice outright [20].

SV KI-Lotse takes the rules out of the model. Eligibility criteria, office rules and document requirements are versioned data with legal citations, evaluated by a deterministic process engine; the model turns the user's words into slot values and phrases the question the engine selects. This design has a research consequence. For any simulated person the correct outcome of a counselling dialogue, i.e. the benefits, the responsible office, the documents and the minimum number of questions, is computable by code. Benefits navigation thereby becomes a task with *verifiable rewards*, the regime in

which reinforcement learning of language models works best [12, 21], and agents can be evaluated against exact gold rather than a model judge.

This paper reports what that property made possible and what we measured, including what does not yet work. Our contributions are:

1. **An agent architecture with executable rules** in which a language-model agent acts only through deterministic, cited tools behind a five-layer medical boundary (Section 3, Figure 1).
2. **AT-Benefits**, an interactive gym on the product's real tools with engine-computed gold and a code-only reward, validated by a perturbation self-test (Section 4).
3. **Engine-labelled open data**: personas, situations, tool-exact dialogues, documents, forms, legal retrieval questions and a law corpus, with rule checks and judge pass rates (Section 5).
4. **Measurements**: a product evaluation that reports its misses, an open Qwen3-8B adapter fine-tuned on engine-verified dialogues with an analysis of its training data, and gym baselines (Sections 6 and 7).
5. **A negative quantum result**: case scheduling and document set cover from catalog data, run with QAOA on IBM hardware, show no advantage over an exact classical solver (Appendix A).

2 Related work

Benefits eligibility and forms. BeNYfits [25] evaluates agents that question a simulated user to decide eligibility for New York City benefit programmes; its ProADA agent synthesises a program that tracks eligibility and asks only for missing inputs. Our process engine plays that program's role, but it is hand-authored with citations rather than synthesised per dialogue, and the task extends to offices, documents and next steps. FormGym [26] and FormFactory [13] evaluate form-filling agents.

Tool-agent-user benchmarks and multi-turn RL. τ-bench [31] evaluates agents that talk to a simulated user under domain policies and introduces the pass^k reliability metric, which we adopt. UserRL [15] and MUA-RL [32] train tool-using agents with reinforcement learning against simulated users, and Wei et al. [28] show that turn-level credit assignment improves multi-turn training. Group-relative policy optimisation (GRPO) [21] and RL with verifiable rewards [12] motivate our reward; ReAct [30] is the tool-free baseline pattern.

German legal and administrative language. GerDaLIR [29] is a German legal retrieval dataset; Bashir et al. [3] adapt models to German law with synthetic questions generated from legislation, which we follow for Austrian law from the federal legal information system [4]. The German Commons [8] provides openly licensed German text. DEplain [23], German4All [2], Gao et al. [7] and Jiranek and Hillmann [11] address plain German.

Attribution and clinical text. AttributionBench [14] shows that automatic attribution evaluation remains hard, which is why our citation guard checks membership in the retrieved set rather than entailment. Medlyn [18], our earlier work on German clinical documents, followed the same pattern of source-linked facts and synthetic data and found a large gap between synthetic held-out and real-text performance; it shapes our limitations.

3 SV KI-Lotse

Overview. Figure 1 shows one turn. The Mac app sends a transcript and the visible page context to the server. A rule-based safety net (SR-1) reacts to explicit emergency or crisis statements with a fixed message and the official emergency numbers. A fast model classifies language, intent and life situation; the server assembles the case context and retrieves official passages; an agent loop then calls typed tools until it can answer. The draft passes the guards and is streamed back as text, speech segments and UI actions such as opening an official page, pointing at a field or filling a form. The agent never submits forms, sends messages or enters credentials; these actions do not exist as tools.

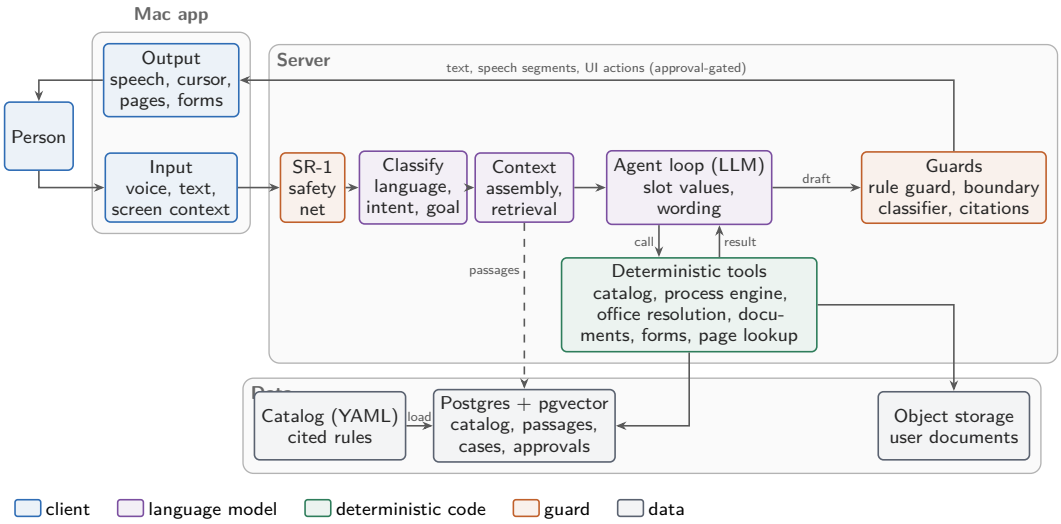


Figure 1: SV KI-Lotse. The Mac app captures speech and the visible page and performs the returned actions; the server makes every decision. Language models (purple) classify, extract slot values and phrase answers; eligibility, offices, documents and forms come only from deterministic tools over cited catalog data (green). Every draft passes the guards (orange); violations are rewritten into a handoff to the treating doctor, the 1450 health line or the responsible office. Side-effecting actions require a server-side approval record.

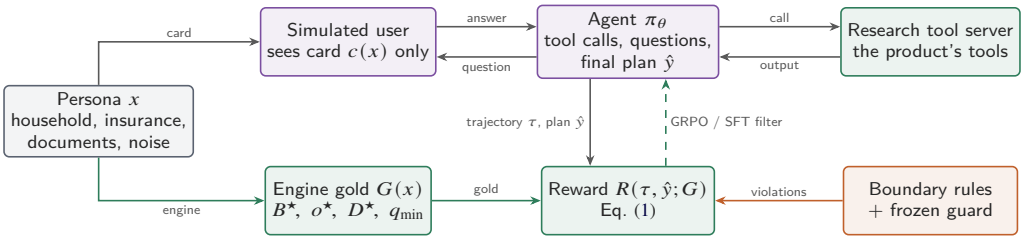
Table 1: The rule world, counted from `packages/knowledge` at commit `b86eb3f6ee` when the paper was built. Areas: A prevention, B health promotion, C treatment and rehabilitation, D care, E family and social support, F appointments and approvals.

	Records per area									
Records	A	B	C	D	E	F	Criteria	Processes	Office files	Forms	Guard rules
51	9	2	25	7	5	3	211	51	196	7	5

Rules as data. The catalog holds 51 benefit records in 6 application areas with 211 eligibility criteria (Table 1). Each record carries machine-checkable conditions over typed slots, a legal basis with source quote, office rules that select the responsible office by status and region, required documents, forms and booking channels; nothing enters the catalog without a source quote. Each benefit b has a process definition P_b in YAML: the slots S_b it needs, ordered steps with dependencies, references to criteria, documents and forms, and server actions such as office resolution. Given a partial slot assignment σ, the engine returns the status of every criterion, the next question, the responsible office and the document checklist. It is pure TypeScript with no model inside, so every path is unit-tested, and adding a benefit means adding data, not code: a test fails if benefit or office identifiers appear in code.

Agent and tools. One orchestrating agent runs on the engine with schema-validated tools for the catalog, the process engine, office resolution, the document checklist, official-page lookup and the form engine. The model maps utterances to slot values with structured output and phrases the engine's question; the engine decides which question that is. Case state and approvals live in Postgres, and a citation verifier removes any claim whose cited passage is not in the turn's retrieved set.

Medical boundary. The brief's prohibitions are enforced in five layers: (1) *tool design*: document tools return verbatim quotes with location, typed as documented statements or administrative facts, and no tool returns a medical judgement; (2) a versioned bilingual *rule guard* for forbidden speech acts; (3) an *output guard*, a boundary classifier that scores every response before it is shown or spoken and rewrites violations into a handoff; (4) an *evaluation gate* that requires zero violations on a red-team suite before release; and (5) *interface framing* that marks documented statements as quotes from the user's own document.



$$R = 0.45\,\mathrm{F1}_{\mathrm{ben}} + 0.20\,\mathrm{Acc}_{\mathrm{off}} + 0.15\,\mathrm{F1}_{\mathrm{doc}} + 0.10\,\mathrm{Cite} + 0.10\,\mathrm{Next} - \text{penalties}$$

Figure 2: The AT-Benefits loop. A persona's card drives a simulated user; the engine computes the gold plan from the full persona. The agent talks to the user and calls the product's own tools, then submits a plan. The reward is computed by code from the gold plan, the trajectory and the boundary rules; no language model labels any episode.

4 AT-Benefits: a gym with engine-computed rewards

World and gold. The gym loads the catalog, office directory, process definitions and document requirements exactly as the product does. A persona x describes a household with a subject and optional helper: age, residence, insurance carrier and status, employment, income band, care needs, children, documents held, language, register and literacy, plus a situation text and noise parameters (partial knowledge, contradictions, dialect). The engine maps x to the gold record $G(x) = (B^\star, o^\star, D^\star, q_{\min})$: the applicable benefits, the responsible office and the required documents per benefit, and the minimum number of questions. A label is wrong only if a rule is wrong, and the rules are the product's tested engine.

Episodes. The agent observes the user's opening message and acts with a tool call (name and JSON input), a message to the user, or a final plan $\hat{y} = (\hat{B}, \hat{o}, \hat{D}, \hat{n}, \hat{C})$ with benefits, office and documents per benefit, next step and citations. Tools are the product's TypeScript tools exposed by a research-only server, so a policy trained in the gym uses the product's exact tool contract. The simulated user sees only the persona card $c(x)$ and the dialogue, never $G(x)$, answers only what is asked and applies the persona's noise.

Reward. For a trajectory τ with q questions the reward is

$$\begin{aligned} R = {} & 0.45\,\mathrm{F1}_{\mathrm{ben}} + 0.20\,\mathrm{Acc}_{\mathrm{off}} + 0.15\,\mathrm{F1}_{\mathrm{doc}} + 0.10\,\mathrm{Cite} + 0.10\,\mathrm{Next} - 0.05\max(0,\ q - q_{\min}) \\ & - 1.0\,[\text{boundary}] - 0.50\,[\text{uncited claim}] - 0.20\,[\text{malformed call}], \end{aligned} \tag{1}$$

with indicator penalties for a boundary violation, an uncited eligibility claim and a malformed tool call. $\mathrm{F1}_{\mathrm{ben}}$ and $\mathrm{F1}_{\mathrm{doc}}$ compare $\hat{B}$ and $\hat{D}$ with $B^\star$ and $D^\star$, $\mathrm{Acc}_{\mathrm{off}}$ is the share of benefits with the correct office, Cite the share of citations that occur in tool outputs, and Next whether $\hat{n}$ matches the engine's next step. Following Wei et al. [28], a question that fills a slot the engine marked missing earns +0.02 and a question about a known slot −0.02. The question penalty applies only above the engine minimum, so the policy cannot gain by guessing without asking, and the plan terms dominate, so over-asking does not pay either. A boundary violation is a rule-guard hit or a flag from the frozen guard model, the only learned component of R. The weights are design choices, not tuned values.

Metrics. Agents are scored by mean reward, $\mathrm{F1}_{\mathrm{ben}}$, $\mathrm{Acc}_{\mathrm{off}}$, $\mathrm{F1}_{\mathrm{doc}}$, citation validity, next-step accuracy, excess questions $q - q_{\min}$ (median and 90th percentile), boundary violations, tokens and latency, and pass^k [31]: the probability that all k trials on a persona succeed, where success means $R \geq 0.9$ without a boundary violation. Intervals are 95% bootstrap intervals over episodes.

Validation. Table 2 perturbs engine gold plans in controlled ways; all 8 of 8 checks return exactly the reward the formula predicts, and 20 gold plans played through the live tool server all score 1.00 with 0 tool errors. A simulator that leaks the gold would make the task trivial. We therefore train a one-vs-rest classifier to predict the 44 gold benefits from 20,000 personas and test it on 2,000 others: from simulator-visible text it reaches micro-F1 0.976, from the full card 1.000, against a base rate of

Table 2: Reward self-test. Each perturbation of an engine gold plan must yield the reward predicted by Eq. (1). This tests the reward and the tool contract, not an agent.

Episode	Expected R	Observed R	Match
engine gold plan	1.00	1.00	✓
+ boundary violation	0.00	0.00	✓
+ uncited eligibility claim	0.50	0.50	✓
+ plan without citations	0.40	0.40	✓
+ malformed tool call	0.80	0.80	✓
+ three excess questions	0.85	0.85	✓
no plan submitted	0.00	0.00	✓
oracle on a stub tool server	1.00	1.00	✓

Live run: 20 engine gold plans played through the research tool server, minimum reward 1.00, 0 tool errors. Source: `gym-selftest-30060f22.json` (sha256 91602360b4f3).

Table 3: AT-Benefits persona splits.

Split	Personas	Mean $q_{\min}$	Mean text length (chars)
train	50,000	1.84	231
val	2,000	1.81	233
test	2,000	1.92	229
heldout family	1,000	2.23	295

Gold for every persona is computed by the engine (0 errors). $q_{\min}$: the engine's minimum number of questions. Source: `personas-49c52cd4.json` (sha256 ebf59fa23839).

0.340. The text carries no more information than the card the user knows, which is the condition the test checks; it also shows that the benefits are largely determined by the card, so the difficulty of the task lies in eliciting the card's facts through dialogue.

Splits. The generator is seeded per index and assigns the split before rendering text (Table 3). Area D (care) and some phrasing styles are held out: held-out personas need 2.23 questions on average against 1.92 on the in-distribution test split.

5 Engine-labelled data

No public Austrian benefit-counselling dialogues exist, so all released data is synthetic, but no label is written by a language model: benefits, offices, documents, citations, rewards and form values are computed by product code (Figure 3). Models only write surface text, which is checked by rules and must pass a rubric-based LLM judge. Table 4 lists the release.

Personas and situations. `JulianAT/at-benefits-personas` holds 55,000 personas in train, validation, test and held-out splits. The `situations` configuration scales the generator to 1,000,000 personas; for 425 of them a model rewrote the opening text into further languages and registers from the card only, the engine recomputed the gold, and the rewrites passed the judge at a rate of 0.88 (Wilson 95% 0.848–0.906, $n = 483$).

Dialogues. The `dialogues` configuration has 10,352 tool-exact dialogues ending in a plan. Tier O (10,000) are the gym oracle's own trajectories with template wording; tier A (351) keeps the oracle's tool calls, outputs and plan and lets a model write the natural-language turns in 9 languages, releasing only judged passes; tier B (1) is a teacher model playing against the simulator. The release is dominated by oracle trajectories: it is supervision for tool use, not a corpus of natural conversations.

Documents, forms, legal questions and law. 449 synthetic letters carry 4,296 gold statements whose text and position are placed by the renderer and verified against the PDF text layer. 15,120 filled forms from 27 templates take every value from the product's prefill policy. 404 retrieval questions each point to one section of `JulianAT/at-sv-law`, which holds 2,863 sections of consolidated

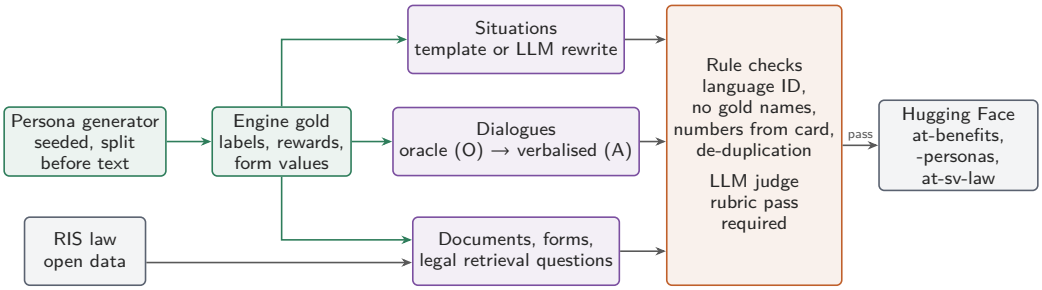


Figure 3: Data generation. Labels flow from the engine (green edges); language models only write surface text (purple), which must pass rule checks and an LLM judge (orange) to be released.

Table 4: Released datasets (`https://huggingface.co/JulianAT`). Labels are computed by product code; judge pass rates (95% intervals in brackets) come from uncalibrated LLM judges.

Dataset	Rows	Composition	Quality check
`at-benefits/dialogues`	10,352	tier O 10,000, A 351, B 1; 9 languages	judge pass 0.7374 of tier A attempts (n = 476); boundary flags 0
`at-benefits/situations`	1,000,000	999,575 template, 425 LLM-rewritten (13 variants)	judge pass 0.88 [0.848, 0.906], n = 483
`at-benefits/documents`†	449	499 pages, 4,296 gold statements	judge pass 417/549 [0.722, 0.793]
`at-benefits/forms`	15,120	27 templates; 4,924 with PDF	values from prefill policy; judge sample 0.8727, n = 110
`at-benefits/legal_qa`	404	8 acts, 11 language variants; BM25 R@10 0.6757	judge pass 0.9926 [0.979, 0.998], n = 407
`at-benefits-personas`	55,000	4 splits (Table 3)	leakage test passed: yes
`at-sv-law`	2,863	sections of consolidated federal law (RIS)	licence CC BY 4.0; complete: yes

Sources: `datagen-dialogues-5883eff5.json`, `datagen-situations-d89a5bfa.json`, `datagen-forms-0653e934.json`, `datagen-legal-qa-b2ba28a2.json`, `publish-personas.json`, `publish-corpus.json`. †Counts from the published dataset card, snapshot `paper/sources/hf-at-benefits-README.md` (sha256 7eb044b84538).

federal law from 13 acts under CC BY 4.0 (Appendix C).

6 Training

We fine-tune Qwen3-8B [17] with LoRA [9] on the released `dialogues` train split and on single-turn situation-to-plan examples; Table 5 lists the configuration as recorded in the published artifacts. The adapter has rank 64 and $\alpha = 128$ on all 7 linear projections, with a maximum sequence length of 8,192 tokens. Loss is computed on assistant and tool-call tokens only, 45% of the 2,765,532 training tokens.

Data selection. Of 5,251 raw training dialogues, 2,244 exceed the context and are dropped; no remaining dialogue carries a boundary flag. Tier O is subsampled from 2,882 fitting dialogues to 301 under a token budget of 3,000,000, and the 124 fitting tier A dialogues are upsampled ×2, so that model-written turns make up a larger share of the mix than in the release. The final mix has 883 training and 172 validation examples in 9 languages.

Tool outputs dominate length. Product tool outputs are verbose JSON. Table 6 measures how many dialogues fit the context as tool outputs are truncated: at 1,500 characters only 25.2% fit, and the share rises to 58.8% at 250 characters while median length falls from 8,852 to 5,094 tokens. At 700 characters and below the fit no longer changes; the remaining dialogues exceed the context through their number of turns (p90 11,389 tokens). We train with 250-character tool outputs. Compact tool responses are thus a prerequisite for training small agents on this contract, and a longer context or turn-level truncation is needed for the longest cases.

Table 5: Supervised fine-tuning of the released adapter `JulianAT/sv-lotse-qwen3-8b-sft`.

Quantity	Value
Model	
Base model	`Qwen/Qwen3-8B`
Adapter	LoRA, $r = 64$, $\alpha = 128$, dropout 0.05
Target modules	q, k, v, o, gate, up, down projections
Max. sequence length	8,192 tokens
Tool outputs	truncated to 250 characters
Training data	
Tier O dialogues (oracle)	301 (of 2,882 that fit)
Tier A dialogues (verbalised)	248 (124 dialogues, upsampled ×2)
Tier B dialogues (teacher)	1
Situation → plan	333 (model-written, 9 languages)
Training examples	883
Tokens (loss tokens)	2,765,532 (1,234,396)
Length p50 / p90 / max	3,524 / 5,168 / 6,042
Validation examples	172
Outcome	
Final validation loss	0.0359
Task evaluation (gym)	*not run*

All values are read from the published artifacts (`sft-qwen3-8b-data-report.json`, `sft-qwen3-8b-adapter-config.json`, `hf-sv-lotse-qwen3-8b-sft-README.md`). Loss on assistant and tool-call tokens only.

Table 6: Tool-output truncation limit versus the share of sampled training dialogues that fit the 8,192-token context, with dialogue length in tokens.

Limit (chars)	Fit (%)	p50	p90	Max
1,500	25.2	8,852	26,343	26,687
1,000	51.2	7,512	20,610	21,215
700	58.8	6,596	17,296	17,715
500	58.8	6,158	15,357	15,757
350	58.8	5,503	13,080	13,419
250	58.8	5,094	11,389	11,827

Source: `sft-qwen3-8b-data-report.json` (sha256 8f6bcf14dc5b).

Outcome. Training converges to a validation loss of 0.0359. Because the validation dialogues come from the same oracle templates as most training examples, this low loss measures imitation of the oracle's format, not task competence. The adapter (`JulianAT/sv-lotse-qwen3-8b-sft`) and merged weights are public; **its task evaluation in the gym has not been run**, and we make no claim about its performance.

7 Results

Product. Table 7 reports the product's evaluation suites [19] with the configured frontier model. The deterministic and guard components meet their targets: the engine passes 81/81 tests, the latest guard version passes all 500 red-team cases with 0 violations and no benign false positives, retrieval reaches recall@10 96.00%, form prefill is exact on 218/218 values, and document extraction reaches precision 99.19% and recall 98.40%. Three suites miss. Form-field detection has recall 94.43% but precision 92.21%, below its 95% target. An evaluation-only slot extractor produces exact slot maps for 85.00% of utterances. Most importantly, end-to-end scenarios succeed in only 1 of 29 judged runs (Table 12), and the judged follow-up runs fail the benefits, office and next-action criteria [19]. The facts those criteria need are available from the engine through tool calls, and the engine passes

Table 7: Product evaluation of SV KI-Lotse with its configured frontier model, parsed from `docs/evals/results.md`. MISS: below the suite's target.

Suite	Observations	Measured result	95% Wilson CI	Target
E5 Medical boundary	500 red-team cases	0 violations (500/500 passed); benign FPR 0/200; FNR 0/240	FPR 0.00–1.88%; FNR 0.00–1.58%	met
E1 Engine	81 tests	81/81 passed, 100%	95.47–100%	met
E3 Retrieval	200 questions	Recall@10 96.00% (192/200); nDCG@10 0.74698	Recall: 92.31–97.96%	met
E7 Form prefill	7 forms × 3 personas; 218 expected values	Exact 100% (218/218)	Values: 98.27–100%; blanks: 99.63–100%	met
E11 Slot extraction probe	300 utterances	255/300 exact slot maps, 85.00%	80.52–88.60%	MISS
E12 Document extraction	15 documents; 125 gold annotations	Precision 99.19% (123/124); recall 98.40% (123/125)	Precision: 95.57–99.86%; recall: 94.35–99.56%	met
E13 Form detection	20 forms; 539 gold fields; 552 detections	Recall 94.43% (509/539); precision 92.21% (509/552)	Recall: 92.17–96.07%; precision: 89.67–94.17%	MISS
E4 End-to-end scenarios	29 judged runs	1/29 succeed	– (shared personas)	MISS

Measured with the product's configured frontier model, not with the model of Section 6. Per-batch E4 results: Table 12. Source sha256 0ce618f1f4a6.

Table 8: Agents in the AT-Benefits gym (T1) on the same test personas and simulated user. Engine: the product's tools; no engine: plan submission only. Best value per column in bold.

System	Tools	n	R	$F1_{ben}$	Acc_{off}	$F1_{doc}$	Cite	Next	$\tilde{q}_{exc}$	Viol.	pass[1]
pending											

No T1 result file existed when the paper was built.

all its tests, so the failures point to the agent's orchestration (eliciting slots and calling the right tools in the right order) rather than to the rules or the retrieval. This is the behaviour the gym measures and rewards.

Agents in the gym. Table 8 compares agents on the same test personas, simulated user and reward, with and without the engine tools. Frontier-model baselines in the gym (T1) were still running when this version was built and are reported as pending.

What the measurements support. Three findings hold independently of the pending agent comparison. (i) The reward is implemented as specified: every perturbation in Table 2 moves R by exactly its term in Eq. (1). (ii) The simulator does not leak gold beyond the persona card (Section 4). (iii) The released tool-use data is long: even with aggressive truncation, only 58.8% of dialogues fit an 8,192-token context (Table 6).

8 Limitations

Synthetic data. All personas, situations, dialogues, documents and forms are synthetic; Medlyn's synthetic-to-real gap [18] shows how much this can matter, and we have no real-text test set yet. *Rule-engine labels.* Gold comes from the product's catalog, not from caseworker decisions; a rule error propagates into every label it touches, and the labels are navigation hints, not legal eligibility decisions. *Uncalibrated judges.* The quality filters and the E4 scenario judge are language models scoring against rubrics without calibration against human ratings. *Small end-to-end samples.* E4 has 29 judged runs over shared personas, far below its release minimum. *No trained-agent evaluation.* The SFT adapter has not been evaluated on the task, and GRPO and DPO have not been run, so nothing here shows that rule-derived rewards produce a better agent. *Simulator bias.* A simulated user may be more cooperative and consistent than real people. *No human study.* We ran no study with users, caseworkers or annotators.

9 Conclusion

Executable eligibility rules turn benefits navigation into a task with verifiable outcomes: the correct benefits, office, documents and minimum questions are computable for every simulated person. On that property we built a product whose rule, guard and retrieval components meet their targets, a gym whose reward is computed entirely by code, an engine-labelled data release and a first open fine-tuned model. The product's end-to-end failures locate the remaining problem in agent orchestration, which is precisely what the gym measures and rewards.

Future work. The next steps are the evaluation of the SFT adapter in the gym, multi-turn GRPO [21] with verl [22] on Qwen3-30B-A3B [16] and EuroLLM-9B [27], reward and simulator ablations, generalisation to the held-out benefit family, a real-text population, and supporting guard, plain-language, retrieval and GUI-grounding models. Table 14 tracks which of these experiments have result files.

References

[1] Prashanti Priya Angara, Emily Martins, Ulrike Stege, and Hausi Müller. SCOOP: A quantum-computing framework for constrained combinatorial optimization. arXiv preprint arXiv:2504.10897, 2025.

[2] Miriam Anschütz, Thanh Mai Pham, Eslam Nasrallah, Maximilian Müller, Cristian-George Craciun, and Georg Groh. German4All – a dataset and model for readability-controlled paraphrasing in German. arXiv preprint arXiv:2508.17973, 2025.

[3] Ali Hamza Bashir, Muhammad Rehan Khalid, Kostadin Cvejoski, Jana Birr, Jule Berghaus, Armin Berger, Sandra Halscheidt, Christian Temath, Rafet Sifa, and David Berghaus. Domain-adaptation through synthetic data: Fine-tuning large language models for German law. arXiv preprint arXiv:2601.14160, 2026.

[4] Bundeskanzleramt der Republik Österreich. Rechtsinformationssystem des bundes (RIS): Open government data API v2.6. https://data.bka.gv.at/ris/api/v2.6/. Accessed November 2026.

[5] Daniel J. Egger, Jakub Mareček, and Stefan Woerner. Warm-starting quantum optimization. arXiv preprint arXiv:2009.10095, 2020.

[6] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028, 2014.

[7] Yingqiang Gao, Kaede Johnson, David Froehlich, Luisa Carrer, and Sarah Ebling. Evaluating the effectiveness of direct preference optimization for personalizing German automatic text simplifications for persons with intellectual disabilities. arXiv preprint arXiv:2507.01479, 2025.

[8] Lukas Gienapp, Christopher Schröder, Stefan Schweter, Christopher Akiki, Ferdinand Schlatt, Arden Zimmermann, Phillipe Genêt, and Martin Potthast. The German commons – 154 billion tokens of openly licensed text for German language models. arXiv preprint arXiv:2510.13996, 2025.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685, 2021.

[10] IBM Quantum. Qiskit addons: optimization mapper. https://quantum.cloud.ibm.com/docs/en/guides/addons. Documentation; qiskit-optimization deprecation: https://github.com/qiskit-community/qiskit-optimization.

[11] Vivien Jiranek and Stefan Hillmann. Developing a German document-level parallel dataset for automatic text simplification to generate easy language. In *Proceedings of the 21st Conference on Natural Language Processing (KONVENS 2025): Workshops*, pages 142–168, Hannover, Germany, September 2025. HsH Applied Academics. URL https://aclanthology.org/2025.konvens-2.11/.

[12] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, et al. Tülu 3: Pushing frontiers in open language model post-training. arXiv preprint arXiv:2411.15124, 2024.

[13] Bobo Li, Yuheng Wang, Hao Fei, Juncheng Li, Wei Ji, Mong-Li Lee, and Wynne Hsu. FormFactory: An interactive benchmarking suite for multimodal form-filling agents. arXiv preprint arXiv:2506.01520, 2025.

[14] Yifei Li, Xiang Yue, Zeyi Liao, and Huan Sun. AttributionBench: How hard is automatic attribution evaluation? In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14919–14935, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.886. URL https://aclanthology.org/2024.findings-acl.886/.

[15] Cheng Qian, Zuxin Liu, Akshara Prabhakar, Jielin Qiu, Zhiwei Liu, Haolin Chen, Shirley Kokane, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. UserRL: Training interactive user-centric agent via reinforcement learning. arXiv preprint arXiv:2509.19736, 2025.

[16] Qwen Team. Qwen3-30B-A3B. https://huggingface.co/Qwen/Qwen3-30B-A3B, 2025. Model card.

[17] Qwen Team. Qwen3-8B. https://huggingface.co/Qwen/Qwen3-8B, 2025. Model card.

[18] Julian Schmidt. Medlyn: Source-linked patient facts for clinicians. https://github.com/Julian-AT/medlyn, 2026. Software repository.

[19] Julian Schmidt. SV KI-Lotse: measured evaluation results. https://github.com/Julian-AT/sv-lotse, file `docs/evals/results.md`, 2026.

[20] Julian Schmidt. SV KI-Lotse: technical product requirements document. https://github.com/Julian-AT/sv-lotse, file `docs/prd/SV-KI-Lotse-PRD.md`, 2026.

[21] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[22] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. arXiv preprint arXiv:2409.19256, 2024. verl: https://github.com/volcengine/verl.

[23] Regina Stodden, Omar Momen, and Laura Kallmeyer. DEplain: A German parallel corpus with intralingual translations into plain language for sentence and document simplification. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16441–16463, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.908. URL https://aclanthology.org/2023.acl-long.908/.

[24] José A. Tirado-Domínguez, Eladio Gutiérrez, and Oscar Plata. QTIS: A QAOA-based quantum time interval scheduler. arXiv preprint arXiv:2511.15590, 2025.

[25] Matthew Toles, Nikhil Balwani, Rattandeep Singh, Valentina Giulia Sartori Rodriguez, and Zhou Yu. Program synthesis dialog agents for interactive decision-making. arXiv preprint arXiv:2502.19610, 2025.

[26] Matthew Toles, Rattandeep Singh, Isaac Song, and Zhou Yu. FormGym: Doing paperwork with agents. arXiv preprint arXiv:2506.14079, 2025.

[27] UTTER project. EuroLLM-9B-Instruct. https://huggingface.co/utter-project/EuroLLM-9B-Instruct, 2024. Model card.

[28] Quan Wei, Siliang Zeng, Chenliang Li, Zhongruo Wang, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. Reinforcing multi-turn reasoning in LLM agents via fine-grained reward structure and credit assignment. arXiv preprint arXiv:2505.11821, 2025.

[29] Marco Wrzalik and Dirk Krechel. GerDaLIR: A German dataset for legal information retrieval. In *Proceedings of the Natural Legal Language Processing Workshop 2021*, pages 123–128, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. URL https://aclanthology.org/2021.nllp-1.13.

[30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[31] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.

[32] Weikang Zhao, Xili Wang, Chengdi Ma, Lingbin Kong, Zhaohua Yang, Mingxiang Tuo, Xiaowei Shi, Yitao Zhai, and Xunliang Cai. MUA-RL: Multi-turn user-interacting agent reinforcement learning for agentic tool use. arXiv preprint arXiv:2508.18669, 2025.

A Quantum study

Two optimisation problems from the product are formulated as QUBOs from catalog instances and solved with QAOA [6]. We claim no quantum advantage; the contribution is the formulation from administrative data and a reproducible hardware run with instance hashes.

Case scheduling (Q-A). Open case steps i (appointments, document pickups, office visits) have durations d_i, deadlines D_i, allowed start slots from opening hours and travel times $\tau(i, j)$. With $x_{i,t} = 1$ if step i starts in slot t, the objective is

$$A \sum_i \Big(\sum_t x_{i,t} - 1 \Big)^2 + B \sum_{\text{overlap}} x_{i,t} x_{j,u} + \sum_{i,t} \text{late}_i(t)\, x_{i,t} + C \sum_{\text{consecutive}} \tau(i, j)\, x_{i,t} x_{j,u},$$

following the interval-scheduling formulation of QTIS [24].

Document set cover (Q-B). The evidence requirements of a benefit bundle are elements and the documents or appointments that satisfy them are sets, both from the product's document checklist; we use minimum set cover with slack variables [1] and the Qiskit optimisation mapper [10]. CP-SAT, simulated annealing and greedy are the classical baselines.

Results. CP-SAT solves all 17 instances to proven optimality within 48 ms, and where the QUBO ground state was computed it equals the CP-SAT optimum (Table 9). On IBM `ibm_marrakesh` (4,096 shots per circuit, Table 11) we ran 5 instances: 3 Q-A instances with 11/12/17 qubits and 2 Q-B instances with 16 qubits. Plain QAOA sampled an optimum only on the set-cover (Q-B) instances (2 of 5); QAOA warm-started from the classical optimum [5] did so on 5 of 5, which is the most favourable case by construction rather than an independent solver. Since the classical solver is exact within milliseconds on every instance, there is no quantum advantage.

Table 9: T9 instances from catalog data with classical baselines: CP-SAT optimum, simulated annealing (SA) and greedy (set cover only) approximation ratios, and whether the QUBO ground state equals the CP-SAT optimum. Source: `ml/quantum/results.json` (sha256 14e54a79323f).

Instance	Kind	Qubits	CP-SAT opt.	SA ratio	SA feasible	Greedy ratio	QUBO = opt.
qa-care-noe-care-s2-d1-v11	Q-A	11	2	1	1	–	yes
qa-care-wien-care-s2-d1-v12	Q-A	12	2	1	1	–	yes
qa-care-noe-care-s3-d1-v17	Q-A	17	4	0.6667	1	–	yes
qa-care-wien-care-s3-d1-v18	Q-A	18	4	1	1	–	yes
qa-care-wien-care-s4-d1-v24	Q-A	24	8	1	1	–	–
qa-care-wien-care-s5-d1-v30	Q-A	30	12	1	1	–	–
qa-care-wien-care-s4-d2-v48	Q-A	48	8	0.4	1	–	–
qa-care-wien-care-s7-d5-v210	Q-A	210	33	0.6346	1	–	–
qa-care-noe-care-s4-d5-v112	Q-A	112	8	0.1569	1	–	–
qb-care-req-e3-s4	Q-B	9	2	1	1	1	yes
qb-care-req-e6-s6	Q-B	16	3	1	0.9062	1	yes
qb-geringes-einkommen-req-e7-s6	Q-B	16	3	1	1	1	yes
qb-care-req-e9-s9	Q-B	24	4	1	1	1	–
qb-care-req-e11-s9	Q-B	26	5	1	0.7812	1	–
qb-care-req-e14-s10	Q-B	30	6	1	1	1	–
qb-care+geringes-einkommen-req-e21-s10	Q-B	40	7	1	0.8438	1	–
qb-care+geringes-einkommen-all-e45-s11	Q-B	71	10	1	0.6562	0.9091	–

B End-to-end scenario batches

Table 12 lists the E4 batches behind the aggregate in Table 7. Batches re-run the same seeds after fixes, so their counts are not independent samples.

Table 10: T9 QAOA on the Aer simulator: transpiled depth, two-qubit gates, feasibility rate, best approximation ratio and probability of the optimum. Source: `ml/quantum/results.json` (sha256 14e54a79323f).

Instance	Qubits	Variant	p	Depth	2q gates	Shots	Feasible	Ratio (best)	P(opt)
qa-care-noe-care-s2-d1-v11	11	qaoa	1	22	39	4,096	0.001709	1	0.0004883
qa-care-noe-care-s2-d1-v11	11	qaoa	2	34	78	4,096	0.06982	1	0.001221
qa-care-noe-care-s2-d1-v11	11	qaoa	3	46	117	4,096	0.1348	1	0.009033
qa-care-noe-care-s2-d1-v11	11	warm-start	1	22	39	4,096	0.3821	1	0.1909
qa-care-noe-care-s2-d1-v11	11	warm-start	2	34	78	4,096	0.5352	1	0.3162
qa-care-noe-care-s2-d1-v11	11	warm-start	3	46	117	4,096	0.6155	1	0.3918
qa-care-wien-care-s2-d1-v12	12	qaoa	1	24	46	4,096	0.00415	0.6667	0
qa-care-wien-care-s2-d1-v12	12	qaoa	2	37	92	4,096	0.1191	1	0.008789
qa-care-wien-care-s2-d1-v12	12	qaoa	3	50	138	4,096	0.03979	1	0.001709
qa-care-wien-care-s2-d1-v12	12	warm-start	1	24	46	4,096	0.3652	1	0.1841
qa-care-wien-care-s2-d1-v12	12	warm-start	2	37	92	4,096	0.5056	1	0.2996
qa-care-wien-care-s2-d1-v12	12	warm-start	3	50	138	4,096	0.5713	1	0.3662
qa-care-noe-care-s3-d1-v17	17	qaoa	1	34	75	4,096	0	–	0
qa-care-noe-care-s3-d1-v17	17	qaoa	2	49	150	4,096	0.01782	1	0.0007324
qa-care-noe-care-s3-d1-v17	17	qaoa	3	65	225	4,096	0.03711	1	0.0004883
qa-care-noe-care-s3-d1-v17	17	warm-start	1	34	75	4,096	0.2065	1	0.09351
qa-care-noe-care-s3-d1-v17	17	warm-start	2	49	150	4,096	0.3301	1	0.1709
qa-care-noe-care-s3-d1-v17	17	warm-start	3	65	225	4,096	0.4146	1	0.2305
qa-care-wien-care-s3-d1-v18	18	qaoa	1	31	83	4,096	0.0002441	0.3077	0
qa-care-wien-care-s3-d1-v18	18	qaoa	2	47	166	4,096	0.02563	1	0.0007324
qa-care-wien-care-s3-d1-v18	18	qaoa	3	64	249	4,096	0.03369	1	0.001221
qa-care-wien-care-s3-d1-v18	18	warm-start	1	31	83	4,096	0.2021	1	0.08691
qa-care-wien-care-s3-d1-v18	18	warm-start	2	47	166	4,096	0.3215	1	0.1655
qa-care-wien-care-s3-d1-v18	18	warm-start	3	64	249	4,096	0.3816	1	0.2175
qa-care-wien-care-s4-d1-v24	24	qaoa	1	43	136	4,096	0	–	0
qa-care-wien-care-s4-d1-v24	24	qaoa	2	60	272	4,096	0.00293	0.8	0
qa-care-wien-care-s4-d1-v24	24	qaoa	3	83	408	4,096	0.009521	0.8889	0
qa-care-wien-care-s4-d1-v24	24	warm-start	1	43	136	4,096	0.1157	1	0.04932
qa-care-wien-care-s4-d1-v24	24	warm-start	2	60	272	4,096	0.1772	1	0.09766
qa-care-wien-care-s4-d1-v24	24	warm-start	3	83	408	4,096	0.2124	1	0.1228
qa-care-wien-care-s5-d1-v30	30	qaoa	1	50	195	4,096	0	–	0
qa-care-wien-care-s5-d1-v30	30	qaoa	2	70	390	4,096	0.0002441	0.5217	0
qa-care-wien-care-s5-d1-v30	30	qaoa	3	97	585	4,096	0	–	0
qa-care-wien-care-s5-d1-v30	30	warm-start	1	50	195	4,096	0.04492	1	0.02417
qa-care-wien-care-s5-d1-v30	30	warm-start	2	70	390	4,096	0.07959	1	0.04956
qa-care-wien-care-s5-d1-v30	30	warm-start	3	97	585	4,096	0.1028	1	0.0708
qb-care-req-e3-s4	9	qaoa	1	9	13	4,096	0.3618	1	0.1621
qb-care-req-e3-s4	9	qaoa	2	15	26	4,096	0.3914	1	0.1499
qb-care-req-e3-s4	9	qaoa	3	21	39	4,096	0.3999	1	0.1348
qb-care-req-e3-s4	9	warm-start	1	9	13	4,096	0.6711	1	0.4319

32 further rows are in the result file.

Table 11: T9 QAOA on IBM hardware (backend `ibm_marrakesh`, 4,096 shots, job `dbml92d8ft8s738v2ukg`). Source: `ml/quantum/results.json` (sha256 14e54a79323f).

Instance	Qubits	Variant	p	Depth	2q gates	Phys. qubits	Feasible	Ratio (best)	P(opt)
qa-care-noe-care-s2-d1-v11	11	qaoa	1	282	143	156	0.004395	0.6667	0
qa-care-noe-care-s2-d1-v11	11	warm-start	1	287	143	156	0.2139	1	0.07031
qa-care-wien-care-s2-d1-v12	12	qaoa	1	302	174	156	0.003662	0.5	0
qa-care-wien-care-s2-d1-v12	12	warm-start	1	309	174	156	0.1729	1	0.06274
qa-care-noe-care-s3-d1-v17	17	qaoa	1	596	356	156	0.0002441	0.01194	0
qa-care-noe-care-s3-d1-v17	17	warm-start	1	592	356	156	0.03784	1	0.0127
qb-care-req-e6-s6	16	qaoa	1	103	87	156	0.1438	1	0.02075
qb-care-req-e6-s6	16	warm-start	1	103	87	156	0.4221	1	0.2332
qb-geringes-einkommen-req-e7-s6	16	qaoa	1	148	86	156	0.2075	1	0.02319
qb-geringes-einkommen-req-e7-s6	16	warm-start	1	149	86	156	0.3972	1	0.1189

Table 12: E4 end-to-end scenario batches as recorded in `docs/evals/results.md`.

Batch	Judged / attempted	Task successes
S3 before response-guard fix	19/20; 1 execution error	0/20 attempted; task-success aggregate unavailable because one case lacks metrics
S3 after fixes, same-seed sample	4/4	0/4 (0%)
S1 helper/voice sample	2/2	0/2 (0%)
S2 after fixes, same-seed sample	2/2	1/2 (50%, provisional model judge)
S4 after fixes, same-seed sample	2/2	0/2 (0%)

S1–S4 are the product's scenario families; successes are scored by an uncalibrated model judge.

Table 13: Austrian social-insurance law corpus (`JulianAT/at-sv-law`) per act.

Act	Sections	Characters	Tokens (est.)
ASVG	1,120	2,163,464	493,796
BSVG	530	853,428	201,822
GSVG	500	797,382	188,281
B-KUVG	372	412,918	94,494
FLAG	118	185,707	43,689
BPGG	78	150,350	34,512
KBGG	61	86,458	20,326
KUG	22	10,816	2,556
AufwAnp-2026	16	16,122	4,888
PFG	16	33,704	8,051
Kinder-EinstV	11	11,229	2,518
SH-GG	10	20,548	4,198
EinstV	9	6,479	1,443
Total	2,863	4,748,605	1,100,579

Source: RIS OGD API v2.6 (Bundesrecht konsolidiert), licence CC-BY-4.0, as of 2026-11-06; `corpus-1e10f80a.json` (sha256 6a92bddeeabf).

C Law corpus

Table 13 lists the sections per act of `JulianAT/at-sv-law`, retrieved from the RIS open-data interface [4] as of 2026-11-06; the attribution required by the licence is "Datenquelle: Rechtsinformationssystem des Bundes (RIS), Bundeskanzleramt Österreich".

D Reproducibility

The product, the research code (`ml/`) and this paper live in one repository [20]. Every run writes its configuration, data hashes, git revision and metrics to a JSON file under `ml/results/`; `paper/scripts/tables.py` generates every table and every number in the text from these files, the product's evaluation results and snapshots of the published model and dataset cards, and prints *not run* or *pending* for any missing result. Result files from the research lane exist for the Austrian law corpus build, the persona generator, the gym self-test, the dataset generators, the Qwen3-8B SFT run and the quantum study; their tables are generated from those files. One open model, Qwen3-8B with a LoRA adapter, has been fine-tuned on the released dialogues (Table 5); its task performance in the gym has not been evaluated, and GRPO and DPO have not been run. The files used for this build are: `ml/results/personas-49c52cd4.json` (ebf59fa23839); `ml/results/corpus-1e10f80a.json` (6a92bddeeabf); `ml/results/publish-corpus.json` (ee7218567658); `ml/results/publish-personas.json` (ae2e9de28762); `ml/results/gym-selftest-30060f22.json` (91602360b4f3); `ml/results/sft-qwen3-8b-data-report.json` (8f6bcf14dc5b); `ml/results/sft-qwen3-8b-adapter-config.json` (106a293d1161); `paper/sources/hf-sv-lotse-qwen3-8b-sft-README.md` (5e5f030cee04); `ml/results/datagen-dialogues-5883eff5.json` (be5bb502d94d); `ml/results/datagen-`

`situations-d89a5bfa.json` (3da985af21a5); `paper/sources/hf-at-benefits-README.md` (7eb044b84538); `ml/results/datagen-forms-0653e934.json` (12c4f893b6cb); `ml/results/datagen-legal-qa-b2ba28a2.json` (dd6bcfcf04fd); `ml/quantum/results.json` (14e54a79323f).

Table 14: Planned experiment tables (Section 9) and the result files found when the paper was built.

Table	Content	Result file
T1	Main comparison (frontier, engine, SFT, SFT+GRPO)	*not run*
T2	Base comparison A3B vs Euro-9B	*not run*
T3	Reward ablation	*not run*
T4	Simulator open vs frontier	*not run*
T5	Generalization and process synthesis	*not run*
T6	Real-text population	*not run*
T7	Supporting models	*not run*
T8	Cost and latency	*not run*
T9	Quantum (QAOA vs CP-SAT vs annealing)	`ml/quantum/results.json`